

COURSEWORK 1

F29AI - Artificial Intelligence and Intelligent Agents

Search and Automated Planning

GROUP 41

Ahlam Rafati H00409464

Syed Taha Rahman H00402356

Table of Contents

Coursework Parts	Page Number
Part 1A	3
Part 1B	4
Part 2A	7
Part 2B	7
Part 2C	8
Output of Missions	10
References	13
Video link	13

Part 1: Solving and Analyzing Sudoku with Search Algorithms

Part 1A

Define Sudoku formally as a constraint satisfaction problem:

Sudoku can be described as a Constraint Satisfaction Problem (CSP) (Russel & Norvig, 2021). Each cell in the 9x9 grid is a variable that must take a value from 1 to 9. The domain of each variable is {1 to 9}, except for the prefilled cells, which have only one possible value. The main constraints are that every row, every column, and every 3x3 subgrid must contain all numbers from 1 to 9 without repetition.

What are the variables, domains, and constraints?

Variables :

Each cell in the 9x9 Sudoku grid is a variable. There are 81 variables in total, one for each position on the board.

Domains:

The domain of each variable is the set of numbers from 1 to 9. If a cell already has a number given in the puzzle, then its domain is only that single value.

Constraints :

The constraints are the rules that must be followed:

- 1-Each row must contain all digits from 1 to 9 with no repetition.
- 2-Each column must contain all digits from 1 to 9 with no repetition.
- 3-Each 3x3 subgrid must also contain all digits from 1 to 9 without repeating any number.

Discuss the time complexity of brute-force search vs. backtracking in Sudoku.

A brute-force search tries every possible combination of numbers until it finds a correct solution (Kumar, 1992; Dechter, 2003). This method has a very large search space (up to 9^{81} possibilities) and is extremely slow because it does not use any logic to reduce options. A backtracking search, on the other hand, fills one cell at a time and checks if the current partial solution is still valid. If a rule is broken, it goes back (backtracks) and tries a different value. This approach is much faster because it avoids exploring impossible solutions early. When combined with techniques like forward checking or arc consistency, it can solve puzzles efficiently (Mackworth, 1977; Haralick & Elliott, 1980).

Part 1B

Introduction :

Sudoku is a popular puzzle game that can be designed using Constraint Satisfaction Problem (CSP). Each of 81 cells in 9 times 9 grid must be filled by digits from 1 to 9. That means:

- Each row should contain unique digits (1-9)
- Each column should contain unique digits (1-9)
- Each 3x3 subgrid should contain unique digits (1-9)

In this project we implemented **backtracking search with forward checking and the minimum remaining values (MRV)** heuristic. The solver reads from textsude.txt file, solves it and gives the output solution efficiently.

Implementation details:

The implemented method is a backtracking search which selects a cell (unassigned variable) and tries a number that follows sudoku rules based on the current board and then uses forward checking to remove the invalid values from the neighbours and goes deeper only if no conflict is approached.

The functions:

__init__ (self, root):

builds the entire interface (grid, buttons, metrics) and initializes variables to track the puzzle.

bt_mrv_fc(self, grid):

This is the main recursive backtrack solver. It chooses the next empty cell using MRV, trying possible values, checks forward constraints and backtrack when needed.

select_mrv_with_domain(self, grid): this function is responsible to fill the next empty cell. It chooses the cell with the fewest possible values. This is called Minimum Remaining Values (MRV) heuristic.

forward_check_failure(self, grid, r, c):

This function is performing forward checking to prevent from assigning values that leads to dead end. This method is useful because it helps to catch up the bad choices early and does not let the algorithm to go to invalid path.

COMPARISON: CSP Solver vs. A* Search

Our implemented solver (Backtracking Search + Forward Checking+MRV)

This solver is based on Constraint Satisfaction Problem (CSP) where each cell is a variable and the domain of the variables are digits from 1 to 9. The constraints are the standard rules of sudoku, there is no duplications in rows, columns or 3x3 boxes.

This method is a great way for solving constraint satisfaction problems. It avoids exploring unnecessary search and detects the dead ends first.

It is a suitable solver for problems such as sudoku that the goal is to assign values that satisfy constraints.

A* Search

A* is a pathfinding search algorithm which is ordered by backward cost and forward cost.

It models the sudoku problem as:

- State: current partially filled sudoku grid
- Action: filling an empty cell with a valid number
- Goal: Completed and valid sudoku grid

How it works?

A* evaluates states based on the sum of cumulative cost $g(n)$ plus the current heuristic(estimation) cost $h(n)$ (Hart et al., 1968; Russel & Norvig, 2021).

FEATURES	Backtracking + MRV + Forward Checking	A* Search Algorithm
Type of Search	Depth-first, constraint-based	Best-first (graph-based)
Time taken to solve sudoku	Fast- solves puzzles in milliseconds	Slow- due to large branching
Scalability	High scalability	Poor scalability
Efficiency	More efficient	Less efficient

In sudoku backtracking is required, but A* doesn't naturally backtrack, it moves forward with its heuristic which leads to many invalid states. CSP eliminates bad values first. In conclusion, A* is not a good solver for sudoku because it is designed to find the shortest path (e.g. like finding a destination, solving maze) that has a start point and goal point. To solve sudoku, the goal is to assign values while satisfying constraints.

Testing

1. Different Puzzle difficulties

- We tested our solutions by trying different level of difficulties (easy, medium and hard) puzzles by inputting .txt files using the “load from file” button in the GUI.

2. Metrics Tracked

Shown live in the GUI

- Time taken (in milliseconds)
- Number of backtracks
- Number of recursive calls

EASY:

Solver Metrics

Time: 12.12 ms

Backtracks: 0

Recursive Calls: 52

MEDIUM :

Solver Metrics

Time: 250.26 ms

Backtracks: 3092

Recursive Calls: 2926

HARD:

Solver Metrics

Time: 827.92 ms

Backtracks: 9317

Recursive Calls: 8527

3. Visual feedback

The GUI highlights:

- Blue: User loaded the file.
- Green: Auto-filled cells by the solver.
- This helped to verify that the solver filled it correctly.

Part 2A

DOMAIN STRUCTURE

The f29ai-rover domain models autonomous planetary exploration scenario in which rovers are deployed from landers to collect data such as images, scans, samples and then transmit them back to the mission. The domain follows a well-defined type of objects, predicates and actions.

Type of objects:

- Rover: unit that moves, collects data and transmit data.
- Lander: A base for deploying rover and transmitting data.
- Waypoint: location for navigation
- Instrument: tools such as camera, scanner, and drill.
- Target/sample/scan: scientific items for imaging, collecting and scanning

Predicates: The predicates describe the location, equipment, deployment and data.

- Location: show where rovers and landers are. (at ?r ?w), (at-l ?l ?w), (adjacent ?a ?b)
- Equipment: defines each instrument on each rover and what they can do. (on-board ?r ?i), (supports-imaging ?i)
- Deployment: means rover is still inside the lander. (docked ?r ?l)
- Data: tracks what data is collected and transmitted (has-image ?r ?t), (has-scan ?r ?x), (has-sample ?r ?s), (image-sent ?t), (scan-sent ?x), (sample-sent ?s)

Actions : The domain defines rover's 8 actions.

- Deploy: releases a rover from lander to the location
- Drive: moves a rover between adjacent waypoints
- Take-image, perform-scan, collect-sample: gather scientific data
- Transmit-image, transmit-scan, transmit-sample: send data back via a lander

Part 2B

Mission 1 :

One rover (r1) and one lander (l1) operate across five waypoints. The goal is to save an image at w5, save a scan at w3, collect a sample at w1, and send all the data back to the lander.

R1 deploys at w1 → collects sample at w1 → sends the sample to l1 at w1

Drives to w5 → takes image at w1 → drives back to w1 → send the image to l1 at w1.

Drives to w3 → performs a scan → drives back to w1 → send scan data to l1 at w1

Mission 2 :

There are two landers (l1,l2), r1 starts already deployed with l1 at w2, r2 starts undeployed (deploys from l2 at w4).

The goal is to save two images, save two scans and collect two samples from different waypoints and return to the lander.

R1 takes an image at w2 → transmits the image to l1 at w2.

R2 deploys at w4 → performs a scan at w4 → transmits the scan to l2 at w4.

R2 drives to w5 → collects sample s2 → drives to w2 → transmits sample s2 to l1.

R2 drives to w3 → takes image tb → drives to w4 → transmits image tb to l2.

R1 drives to w1 → collects sample s1 → drives through w6 → drives to w4 → transmits sample s1 to l2.

R2 drives to w6 → performs scan x6 → drives back to w2 → transmits scan x6 to l1

Part 2C: Extended Domain

Mission 3 :**DOMAIN STRUCTURE**

The domain was extended to include new types, predicates, and actions representing astronaut roles and lander facilities.

Type of objects:

- Astronaut: represents a human operator (Alice, Bob).
- Facility: represents internal areas within lander.

Predicates:

- (at-a ?a - astronaut ?f - facility) – tracks astronaut location.
- (is-docking-bay ?f) / (is-control-room ?f) – identify facility type.
- (facility-of ?f ?l) – associates each facility with a specific lander.
- (f-adjacent ?f1 ?f2) – defines adjacency between facilities to allow movement.

Actions :

- move-astronaut: enables astronauts to move between adjacent facilities within the same lander.
- deploy: now requires an astronaut to be in the docking bay of the corresponding lander.
- collect-sample: requires an astronaut in a docking bay.
- Transmit-image, transmit-scan, transmit-sample: require an astronaut in the control room to authorize data transfer.

There are two landers (l1, l2), two rovers (r1, r2), and two astronauts (Alice and Bob).

Each lander has two internal areas: a docking bay and a control room.

Alice is stationed in the docking bay of l1, and Bob is in the control room of l2.

A rover can only be deployed or collect a sample when an astronaut is in the docking bay, and data transmission can only occur when an astronaut is in the control room.

The goal is to save two images, two scans, and two samples while respecting the astronaut presence rules.

Alice and Bob can move between the docking bay and control room of their respective landers.

Bob moves from control room → docking bay (l2) → deploys r2 → r2 performs scan x4 at w4 → Bob moves back to control room → r2 transmits scan x4 to l2.

R2 drives to w5 → collects sample s2 → drives back to w4 → transmits sample s2 to l2.

R2 drives to w3 → takes image tb → drives to w4 → transmits image tb to l2.

Alice moves from docking bay → control room (l1) → r1 takes image ta at w2 → transmits image ta to l1.

Alice moves to docking bay → r1 drives to w1 → collects sample s1 → drives back to w2 → Alice moves to control room → r1 transmits sample s1 to l1.

R2 drives to w6 → performs scan x6 → drives back to w4 → Bob moves to control room → r2 transmits scan x6 to l2.

Planner Used

For testing and validating the F29AI-Rover domain and its problem instances, the planner used was Fast Downward.

Fast Downward is a widely used STRIPS-compatible planner that supports typed domains and deterministic planning problems.

Output of Missions

Mission 1:

Found Plan (output)

(deploy r1 l1 w1)

(collect-sample r1 drill1 sa w1)

(transmit-sample r1 l1 sa w1)

(drive r1 w1 w5)

(take-image r1 cam1 ta w5)

(drive r1 w5 w1)

(transmit-image r1 l1 ta w1)

(drive r1 w1 w2)

(drive r1 w2 w3)

(perform-scan r1 scan1 sc w3)

(drive r1 w3 w2)

(drive r1 w2 w1)

(transmit-scan r1 l1 sc w1)

```
(:action deploy
:parameters (r1 l1 w1)
:precondition
  (and
    (docked r1 l1)
    (at-l l1 w1)
  )
:effect
  (and
    (at r1 w1)
    (not
      (docked r1 l1)
    )
  )
)
```

Mission 2:

domain.pddl	Found Plan (output) <pre>(take-image r1 cam1 ta w2)</pre>	<pre>(:action take-image :parameters (r1 cam1 ta w2) :precondition (and (at r1 w2) (on-board r1 cam1) (instrument cam1) (supports-imaging cam1) (target-at ta w2) (free-mem r1)) :effect (and (holding-image r1 ta) (not (free-mem r1))))</pre>
domain-ext.pddl		
mission1.pddl		
mission2.pddl		
mission3.pddl		
Plan (1)		
Plan (2)		
Plan (3)		

Mission 3:

PDDL Editor File - Session - Import Solver Plugins Help

domain.pddl
domain-ext.pddl
mission1.pddl
mission2.pddl
mission3.pddl
Plan (1)
Plan (2)
Plan (3)

Found Plan (output)

```
(take-image r1 cam1 ta w2)
(move-astronaut alice db1 cr1)
(transmit-image r1 l1 ta w2 alice cr1)
(drive r1 w2 w3)
(take-image r1 cam1 tb w3)
(drive r1 w3 w4)
(transmit-image r1 l2 tb w4 bob cr2)
(perform-scan r1 scn1 x4 w4)
(transmit-scan r1 l2 x4 w4 bob cr2)
(drive r1 w4 w5)
(drive r1 w5 w6)
(perform-scan r1 scn1 x6 w6)
(drive r1 w6 w1)
(drive r1 w1 w2)
(transmit-scan r1 l1 x6 w2 alice cr1)
(move-astronaut alice cr1 db1)
(drive r1 w2 w5)
(collect-sample r1 dr1 l1 s2 w5 alice db1)
(drive r1 w5 w4)
(transmit-sample r1 l2 s2 w4 bob cr2)
(drive r1 w4 w5)
(drive r1 w5 w6)
(drive r1 w6 w1)
(collect-sample r1 dr1 l1 s1 w1 alice db1)
(move-astronaut alice db1 cr1)
(drive r1 w1 w2)
(transmit-sample r1 l1 s1 w2 alice cr1)
```

```
(:action take-image
:parameters (r1 cam1 ta w2)
:precondition
  (and
    (at r1 w2)
    (on-board r1 cam1)
    (instrument cam1)
    (supports-imaging cam1)
    (target-at ta w2)
    (free-mem r1)
  )
:effect
  (and
    (holding-image r1 ta)
    (not
      (free-mem r1)
    )
  )
)
```

References:

Dechter, R. (2003) *Constraint Processing*. San Francisco: Morgan Kaufmann.

Haralick, R.M. and Elliott, G.L. (1980) 'Increasing tree search efficiency for constraint satisfaction problems', *Artificial Intelligence*, 14(3), pp. 263–313.

Hart, P.E., Nilsson, N.J. and Raphael, B. (1968) 'A Formal Basis for the Heuristic Determination of Minimum Cost Paths', *IEEE Transactions on Systems Science and Cybernetics*, 4(2), pp. 100–107.

Kumar, V. (1992) 'Algorithms for constraint-satisfaction problems: A survey', *AI Magazine*, 13(1), pp. 32–44.

Mackworth, A.K. (1977) 'Consistency in networks of relations', *Artificial Intelligence*, 8(1), pp. 99–118.

Russell, S.J. and Norvig, P. (2021) *Artificial Intelligence: A Modern Approach*. 4th edn. Hoboken, NJ: Pearson Education.

VIDEO LINK:

<https://screenapp.io/app/v/2ysrFvqAKS>